APRENDIZAJE GUIADO CON GEMINI

ALGORITMO XGBOOST

Prof. Ing. Miguel Camacaro Pérez



Introducción

Este documento ofrece una guía didáctica y profesional sobre el algoritmo *XGBoost*, estructurada para brindar una comprensión profunda de su arquitectura, funcionamiento y aplicaciones prácticas en el modelado de datos tabulares y la valoración inmobiliaria.

El contenido está organizado en dos bloques principales. En primer lugar, un informe técnico examina cómo la construcción secuencial de árboles de decisión permite corregir progresivamente las desviaciones de estimaciones previas. Se destacan capacidades avanzadas como el procesamiento nativo de información faltante mediante rutas predeterminadas y la aceleración en volúmenes masivos de datos a través de la agrupación en intervalos discretos. Adicionalmente, se presenta una guía clara para configurar hiperparámetros que regulan la profundidad del modelo y limitan la complejidad estructural, previniendo fallos de generalización en entornos reales. Se completa este bloque analizando la transparencia predictiva mediante herramientas que descomponen la contribución de cada atributo en la estimación final.

En segundo lugar, se incorporan recursos pedagógicos interactivos pensados para afianzar el conocimiento. Destacan esquemas visuales jerárquicos, tarjetas didácticas de repaso activo, cuestionarios de autoevaluación con respuestas justificadas y planes estructurados de estudio orientados a la práctica continua y la resolución de problemas técnicos en producción

Contenido

Informe Técnico: Aplicación de XGBoost en la Valuación Inmobiliaria 4

1. Introducción y Fundamentos Metodológicos 4

2. El Rol de *XGBoost* en la Arquitectura de Valuación 4

3. Integración en Flujos de Trabajo Híbridos (Evidencia Empírica) 5

4. Explicabilidad del Modelo (XAI) en la Tasación 5

5. Conclusiones y Buenas Prácticas para Valuadores 6

Algoritmo XGBoost en el Modelado Tabular y Valuación Inmobiliaria 6

Parte 1: informe técnico estructurado 6

1. Fundamentos Avanzados y Arquitectura de *XGBoost* 6

2. Innovaciones Clave en Rendimiento y Manejo de Datos 9

3. Guía de Hiperparametrización y Regularización 10

4. Casos de Uso y Diagnóstico de Errores en Producción 12

Parte 2: Recursos interactivos de estudio 14

1. Mapa Mental Conceptual 14

2. Set de Tarjetas Didácticas (Flashcards) 14

3. Cuestionario de Autoevaluación Formativa 16

Prompting: proceso de aprendizaje 18

1. Entender: Algoritmo de *Machine Learning XGBoost* 18

2. Profundizar: Algoritmo de Machine Learning: XGBoost 21

3. Practicar: Algoritmo XGBoost 26

4. Evaluar: Algoritmo XGBoost 30

5. Planificar: Algoritmo XGBoost 36

Referencias Bibliográficas 39

Informe Técnico: Aplicación de XGBoost en la Valuación Inmobiliaria

1. Introducción y Fundamentos Metodológicos

La valuación inmobiliaria masiva (*Automated Valuation Models*, AVM) y la predicción de índices de precios de vivienda (*House Price Index*, HPI) han evolucionado desde los modelos econométricos tradicionales de regresión hedónica (HPM) hacia enfoques avanzados de aprendizaje automático (*Machine Learning*, ML).

Tradicionalmente, los modelos de regresión lineal múltiple y las tasaciones periciales tradicionales se enfrentaban a limitaciones severas para capturar relaciones no lineales complejas entre los atributos intrínsecos de los inmuebles y la dinámica del mercado. En este contexto, los algoritmos basados en árboles de decisión con refuerzo de gradiente (*Gradient Boosting*), y en particular *XGBoost* (*Extreme Gradient Boosting*), se han consolidado como herramientas de alto rendimiento debido a su capacidad superior para modelar interacciones complejas de datos tabulares, espaciales y macroeconómicos.

2. El Rol de *XGBoost* en la Arquitectura de Valuación

XGBoost destaca frente a otros modelos de ensamblaje (como *Random Forest*) porque construye los árboles de decisión de forma secuencial (en cascada), donde cada nuevo árbol se entrena específicamente para corregir los errores o gradientes residuales de los árboles anteriores.

Ventajas Técnicas Clave:

1. Optimización de Segundo Orden: A diferencia del *Gradient Boosting* clásico que solo utiliza derivadas de primer orden, *XGBoost* emplea una expansión en serie de *Taylor* de segundo orden (incorporando gradientes y hessianas). Esto permite evaluar la curvatura de la función de pérdida y calcular de forma analítica y eficiente los pesos óptimos de las hojas.
2. Manejo Nativo de Valores Perdidos (*Sparsity-aware Split Finding*): En bases de datos inmobiliarias masivas extraídas de portales web o registros públicos, es común encontrar campos vacíos (NaN). *XGBoost* maneja estos nulos de manera nativa asignando una dirección por defecto en cada nodo de división para maximizar la ganancia de información, evitando distorsiones por imputaciones estáticas previas.
3. Regularización Incorporada: Incluye penalizaciones por complejidad (términos L1 y L2 sobre los pesos y número de hojas), lo que mitiga drásticamente el sobreajuste (*overfitting*) en mercados con alta volatilidad de precios.

3. Integración en Flujos de Trabajo Híbridos (Evidencia Empírica)

La literatura reciente y la investigación aplicada en ciencia de datos inmobiliarios demuestran que *XGBoost* rinde de manera sobresaliente cuando se integra dentro de arquitecturas metodológicas estructuradas:

- Modelado Multifuente y de Dos Etapas: En estudios recientes aplicados a la predicción de índices inmobiliarios urbanos (como los modelos desarrollados para *Fráncfort del Meno*), se implementan enfoques de dos etapas que combinan datos micro de transacciones y ofertas (*listings*) con variables socioeconómicas y macrofinancieras. En estos sistemas, *XGBoost* se utiliza para estimar con precisión los precios por metro cuadrado a nivel de micro-ubicación, superando ampliamente a los enfoques de persistencia ingenua (*random walk*) con errores porcentuales absolutos medios (MAPE) inferiores al 1,85 %.
- Marcos de Aprendizaje Ágil (*SFMLAF*): Evaluaciones empíricas en grandes mercados (como el análisis de transacciones residenciales en el Reino Unido con cientos de miles de observaciones) sitúan a *XGBoost* con métricas de desempeño sobresalientes, alcanzando errores reducidos (*RMSE* de 0,444 y *MAPE* del 1,94 %), lo que valida su robustez frente a sesgos de información y su alta aplicabilidad para profesionales del sector inmobiliario.
- Enfoques de Ensamble (*Stacking*): En contextos de alta asimetría de información y diversidad de fuentes (como plataformas de anuncios en mercados emergentes o europeos), *XGBoost* suele combinarse con otros algoritmos en estructuras de *stacking*, utilizando metaniveles de regresión (por ejemplo, *Ridge CV*) para fusionar las predicciones y equilibrar la interpretabilidad económica con la potencia predictiva no lineal.

4. Explicabilidad del Modelo (XAI) en la Tasación

Uno de los principales retos históricos de los algoritmos de *Machine Learning* en la valuación inmobiliaria ha sido su naturaleza de "caja negra". Para cumplir con los estándares normativos de la valoración profesional y bancaria, los modelos de *XGBoost* se complementan sistemáticamente con herramientas de Inteligencia Artificial Explicable (XAI), destacando el uso de valores SHAP (*SHapley Additive exPlanations*):

- Atribución de Conductores de Valor: SHAP permite descomponer la estimación de precio de un inmueble específico en contribuciones individuales correspondientes a sus impulsores de valor (superficie útil, antigüedad, calidades constructivas, accesibilidad por transporte y variables macroeconómicas de tipo de interés o inflación).

- Transparencia en la Decisión: Esto permite a tasadores, entidades financieras e inversores auditar por qué el modelo predice un valor determinado, vinculando la precisión estadística del algoritmo con los principios económicos tradicionales de la tasación.

5. Conclusiones y Buenas Prácticas para Valuadores

1. Superioridad Predictiva: XGBoost supera a los modelos lineales tradicionales y a la regresión hedónica clásica al capturar con precisión las non-linealidades espaciales y estructurales del mercado inmobiliario.
2. Mitigación de Fugas de Datos (*Data Leakage*): Al implementar XGBoost en valuación masiva, es imperativo aplicar divisiones agrupadas por propiedad (*grouped splits*), excluir metadatos temporales superpuestos y separar estrictamente las muestras para garantizar la integridad metodológica fuera de muestra.
3. CoHumanAI (Humano en el Bucle): El uso de XGBoost asistido por plataformas analíticas avanzadas no sustituye el criterio pericial del tasador, sino que actúa como un multiplicador intelectual que reduce la subjetividad y optimiza la precisión en la toma de decisiones hipotecarias y de inversión.

Algoritmo XGBoost en el Modelado Tabular y Valuación Inmobiliaria

Parte 1: informe técnico estructurado

1. Fundamentos Avanzados y Arquitectura de *XGBoost*

Aprendizaje Secuencial (*Boosting*) vs. Ensamblado Independiente (*Bagging*)

En el aprendizaje automático sobre datos tabulares, las arquitecturas de ensamble basadas en árboles se dividen fundamentalmente en dos paradigmas: *Bagging* (ej. Random Forest) y *Boosting* (ej. *Gradient Boosting Decision Trees* / *XGBoost*)

- *Bagging* (*Bootstrap Aggregating*): Construye múltiples árboles de decisión de forma independiente y paralela a partir de submuestras aleatorias tomadas con reemplazo de la muestra de entrenamiento (muestreo *bootstrap*). Cada árbol se entrena sin interacción con los demás, y la predicción final se obtiene promediando las salidas individuales en regresión o mediante voto de mayoría en clasificación. El objetivo primordial de *Bagging* es reducir la varianza del modelo sin incrementar apreciablemente el sesgo.
- *Boosting Secuencial* (XGBoost): Opera de manera estrictamente aditiva e iterativa. En lugar de entrenar árboles aislados, *XGBoost* inicializa la estimación con un valor constante

(típicamente la media de la variable objetivo en regresión) y añade secuencialmente K árboles de decisión $f_k \in \mathcal{F}$:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

Donde cada nuevo árbol de decisión f_k se entrena orientándose de forma explícita a corregir los residuos o errores de predicción acumulados por el ensamble acumulado en la iteración previa ($\hat{y}_i^{(t-1)}$). Esta dinámica secuencial focaliza la capacidad de aprendizaje en las observaciones más complejas o con mayores residuos, reduciendo sistemáticamente tanto el sesgo como la varianza.

Optimización de la Función Objetivo y Expansión en Serie de Taylor de Segundo Orden

La predicción final para una observación i con vector de características x_i se expresa como la suma aditiva de K funciones arbóreas:$\hat{y}_i = \sum_{k=1}^{K} f_k\,(x_i)$. En la iteración t, el algoritmo busca incorporar la estructura f_t que minimice la función objetivo regularizada:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l\left(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t)$$

Donde l representa una función de pérdida convexa y diferenciable (como el Error Cuadrático Medio en regresión), y $\Omega(f_t)$ es el término de penalización por complejidad formalizado como:

$$\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 + \alpha \sum_{j=1}^{T} |w_j|$$

En esta formulación, T denota el número total de hojas del árbol f_t, w_j es la puntuación o peso asignado a la hoja j, γ es la constante de penalización por adición de hojas, λ representa el coeficiente de regularización L2 (*Ridge*) y α es el coeficiente L1 (*Lasso*)

A diferencia de los algoritmos clásicos de *Gradient Boosting* (GBM) que utilizan únicamente derivadas de primer orden (gradientes) para aproximar el descenso, *XGBoost* aplica una expansión en serie de *Taylor* de segundo orden sobre la función de pérdida en torno a la estimación previa $\hat{y}_i^{(t-1)}$:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{n} \left[l\left(y_i, \hat{y}_i^{(t-1)}\right) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

Donde las derivadas parciales de primer y segundo orden se definen formalmente como8more_horiz:

- Gradiente de primer orden (g_i):

$$g_i = \frac{\partial l\left(y_i, \hat{y}_i^{(t-1)}\right)}{\partial \hat{y}_i^{(t-1)}}$$

- Matriz/Valor Hessiano de segundo orden (h_i):

$$h_i = \frac{\partial^2 l\left(y_i, \hat{y}_i^{(t-1)}\right)}{\partial \left(\hat{y}_i^{(t-1)}\right)^2}$$

Al eliminar los términos constantes $l\left(y_i, \hat{y}_i^{(t-1)}\right)$ que no dependen de la nueva función f_t, la función objetivo simplificada que se optimiza agrupando las observaciones en el conjunto de índices I_j correspondientes a cada hoja j adopta la forma:

$$\tilde{\mathcal{L}}^{(t)} = \sum_{j=1}^{T}\left[\left(\sum_{i\in I_j} g_i\right)w_j + \frac{1}{2}\left(\sum_{i\in I_j} h_i + \lambda\right)w_j^2\right] + \gamma T$$

Determinación de Pesos Óptimos y Criterio de Ganancia de Partición

Para una estructura arbórea fija, diferenciando $\tilde{\mathcal{L}}^{(t)}$ con respecto al peso de la hoja w_j e igualando a cero, se obtiene la expresión analítica exacta para el peso óptimo de la hoja j **(**w_j^***)**:

$$w_j^* = -\frac{\sum_{i\in I_j} g_i}{\sum_{i\in I_j} h_i + \lambda}$$

Sustituyendo w_j^* en la función objetivo se obtiene la función de estructura mínima de pérdida residual, utilizada como métrica de calidad de la topología del árbol:

$$\tilde{\mathcal{L}}^{(t)}(q) = -\frac{1}{2}\sum_{j=1}^{T}\frac{\left(\sum_{i\in I_j} g_i\right)^2}{\sum_{i\in I_j} h_i + \lambda} + \gamma T$$

Durante la construcción del árbol, el algoritmo evalúa la conveniencia de dividir un nodo hoja en dos sub-hojas (izquierda I_Ly derecha I_R). El beneficio o Ganancia (*Gain*) de dicha partición se calcula como:

$$\text{Gain} = \frac{1}{2}\left[\frac{\left(\sum_{i\in I_L} g_i\right)^2}{\sum_{i\in I_L} h_i + \lambda} + \frac{\left(\sum_{i\in I_R} g_i\right)^2}{\sum_{i\in I_R} h_i + \lambda} - \frac{\left(\sum_{i\in I} g_i\right)^2}{\sum_{i\in I} h_i + \lambda}\right] - \gamma$$

Si la ganancia calculada no supera la penalización por complejidad γ, la partición se descarta mediante poda retroactiva (*pruning*), garantizando que sólo se añada profundidad al árbol si la reducción del error compensa el costo del parámetro γ.

2. Innovaciones Clave en Rendimiento y Manejo de Datos

Búsqueda de Partición Consciente de la Dispersión (*Sparsity-aware Split Finding*)

Los datos tabulares del mundo real —especialmente en registros inmobiliarios y crediticios— presentan una alta frecuencia de valores perdidos (*NaN*) o vectores dispersos debido a registros incompletos. Mientras que los modelos tradicionales requieren técnicas de imputación previa que pueden introducir sesgo o distorsionar distribuciones, *XGBoost* integra un mecanismo nativo:

1. En cada nodo de decisión, se asigna una dirección por defecto (*default direction*) para todos los valores nulos o dispersos.
2. El algoritmo evalúa la Ganancia enviando todos los datos faltantes primero a la rama izquierda y luego a la rama derecha.
3. Mediante el descenso de gradiente, *XGBoost* aprende iterativamente la trayectoria por defecto que maximiza la reducción del objetivo, reteniendo esa regla para la etapa de inferencia.

Métodos Basados en Histogramas (*tree_method = 'hist'*)

Para determinar el corte óptimo en un atributo continuo, los algoritmos exactos (*exact greedy*) deben ordenar y evaluar todas las observaciones, lo que resulta computacionalmente costoso sobre grandes volúmenes de datos. El método basado en histogramas (*tree_method = 'hist'*):

- Agrupa las características continuas en *bins* o contenedores discretos (ej. 256 intervalos).
- Construye histogramas de gradientes (g_i) y hessianos (h_i) acumulados dentro de cada contenedor.
- Reduce la complejidad de búsqueda de puntos de división de $\mathcal{O}(\text{datos} \times \text{características})$ a $\mathcal{O}(\text{bins} \times \text{características})$, acelerando drásticamente los tiempos de entrenamiento y reduciendo el consumo de memoria operativa.

Submuestreo Estocástico y Regularización de Características

Para prevenir que observaciones atípicas o variables dominantes sesguen la estructura del ensamble, *XGBoost* incorpora dos niveles de aleatorización inspirados en *Bagging*:

- Submuestreo de Filas (*subsample*): Selecciona estocásticamente una fracción de las observaciones de entrenamiento para construir cada árbol.
- Submuestreo de Columnas (*colsample_bytree* / *colsample_bylevel*): Selecciona una submuestra aleatoria de variables antes de construir cada árbol o cada nivel de decisión, rompiendo la correlación entre los estimadores base.

3. Guía de Hiperparametrización y Regularización

Rol Operativo de los Hiperparámetros Críticos

Rol Operativo de los Hiperparámetros Críticos

Hiperparámetro	Rol Técnico y Mecanismo Interno	Rango Típico de Ajuste	Impacto en el Modelo
***learning_rate* / eta**	Pondera y escala la contribución marginal de cada nuevo árbol (f_t) al ensamble acumulado.	0,01 – 0,20	Valores menores previenen sobreajuste pero requieren mayor cantidad de estimadores (*n_estimators*).
max_depth	Controla la profundidad máxima de ramificación permitida en cada árbol de decisión.	3 - 10	Valores elevados permiten capturar interacciones complejas, pero incrementan drásticamente el riesgo de sobreajuste.
min_child_weight	Define la suma mínima de la masa del hessiano ($\sum h_i$) requerida dentro de un nodo para autorizar una división.	1 - 20	En regresión con MSE ($h_i = 2$), equivale al número mínimo de observaciones ponderadas por nodo. Previene aislamiento de *outliers*.
***gamma* (γ)**	Umbral mínimo de Ganancia (*Gain*) requerido para realizar una nueva partición.	0,0 – 10,0	Funciona como parámetro directo de poda (*pruning*); cuanto mayor es γ, más conservador resulta el árbol.
***reg_lambda* (λ)**	Coeficiente de penalización por regularización L2 (*Ridge*) sobre los pesos de las hojas w_j.	0,1 – 10,0	Comprime suavemente los pesos de las hojas hacia cero, suavizando las predicciones en regiones con poca densidad de datos.
***reg_alpha* (α)**	Coeficiente de penalización por regularización L1 (*Lasso*) sobre los pesos de las hojas w_j.	0,0 – 10,0	Conduce los pesos de las hojas irrelevantes exactamente a cero, promoviendo modelos más dispersos y robustos.
subsample	Proporción aleatoria del conjunto de datos utilizada para entrenar cada árbol.	0,6 – 0,9	Introduce variabilidad estocástica, previniendo la memorización del conjunto de entrenamiento.
colsample_bytree	Proporción de características seleccionadas al azar al construir cada árbol.	0,6 – 0,9	Reduce la dominancia de atributos extremadamente fuertes e incrementa la diversidad del ensamble

Tabla Diagnóstica de Sobreajuste (*Overfitting*) vs. Subajuste (*Underfitting*)

```
DIAGNÓSTICO DE DESEMPAPEÑO DEL MODELO

SOBREAJUSTE (OVERFITTING)
• Train error << Test error
• R²_train - R²_test > 10% [34]
• Modelo memoriza el ruido

SUBAJUSTE (UNDERFITTING)
• Train error ≈ Test error
• R² extremadamente bajo
• Modelo no capta la señal
```

Métrica / Diagnóstico	Síntomas en Entrenamiento vs. Validación / Test	Causas Raíz Técnicas	Acciones Correctivas en Hiperparámetros de XGBoost
Sobreajuste (*Overfitting*)	• $R^2_{\text{train}} \gg R^2_{\text{test}}$ (Diferencia $> 10\%$) [34] [35]. • $\text{RMSE}_{\text{test}}$ sustancialmente superior a $\text{RMSE}_{\text{train}}$ [35] [36]. • El modelo memoriza ruido atípico [35] [37].	• Árboles con excesiva profundidad (`max_depth` muy alto) [1] [30]. • Ausencia de penalización regularizadora ($\lambda = 0, \alpha = 0, \gamma = 0$) [12] [–]. • Tasa de aprendizaje elevada sin suficiente restricción [30] [31].	1. Reducir `max_depth` (ej. bajar de 10-14 a 3-6) [30] [31]. 2. Incrementar `gamma` (γ) para forzar la poda de ramas improductivas [21] [–]. 3. Aumentar regularización `reg_lambda` (λ) y `reg_alpha` (α) [12] [–]. 4. Ajustar `subsample` y `colsample_bytree` a $0.6 - 0.8$ [31]. 5. Aplicar *Early Stopping* monitoreando la pérdida de validación fuera de muestra [30] [38].
Subajuste (*Underfitting*)	• R^2_{train} y R^2_{test} ambos deficientes (ej. < 0.60) [37]. • Métrica de error (RMSE/MAE) alta en ambos subconjuntos sin discrepancia significativa entre ellos [35] [37].	• Modelo excesivamente restringido o simplificado [30] [37]. • Tasa de aprendizaje muy baja (`eta` < 0.01) con número insuficiente de árboles (`n_estimators`) [30] [31]. • Penalizaciones por regularización o poda desproporcionadamente altas [21] [30].	1. Incrementar `max_depth` para permitir capturar interacciones complejas [30] [31]. 2. Aumentar `n_estimators` y/o ajustar `learning_rate` a valores entre $0.05 - 0.10$ [30] [31]. 3. Reducir los umbrales de poda `gamma` (γ) y `min_child_weight` [21] [–]. 4. Disminuir los coeficientes de regularización L1 y L2 [12] [–].

4. Casos de Uso y Diagnóstico de Errores en Producción

Aplicaciones de Valuación Inmobiliaria Automatizada (AVMs) y Evidencia Empírica

En la literatura científica entre 2024 y 2026, *XGBoost* se ha consolidado como la arquitectura de referencia en Modelos de Valuación Automatizada (AVMs) y estudios econométricos espaciales:

- Rendimiento Predictivo y Escalabilidad: En el mercado del Reino Unido sobre más de 950.000 registros transaccionales, *XGBoost* alcanzó un RMSE de 0,444 y un MAPE de 1.94 %. En Estocolmo, demostró métricas de precisión similares o superiores a *Random Forest*, pero reduciendo el tiempo de entrenamiento entre 2 y 50 veces y el tiempo de inferencia hasta 40 veces. En el conjunto de datos de Ames (Iowa), alcanzó un R^2 de 0,930 y un MSE de 0,001 tras la optimización con *GridSearch*.
- Integración con Preprocesamiento Espacial: La combinación de algoritmos de agrupamiento espacial restringido (SCMCA) con *XGBoost* logró incrementos sustanciales de precisión (R^2 = 0,762, RMSE = 0,056 en submercados) manteniendo la consistencia espacial en tasaciones masivas.

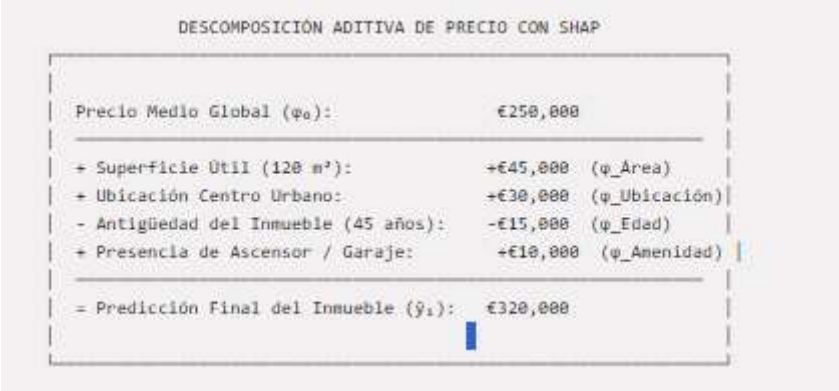
Integración con Inteligencia Artificial Explicable (*SHAP Values*)

A pesar de su elevado poder predictivo, los modelos de *boosting* han sido históricamente etiquetados como "cajas negras". La integración del algoritmo *TreeExplainer* mediante Valores SHAP (*Shapley Additive Explanations*) descompone la predicción aditivamente en unidades monetarias explícitas:

$$\hat{y}_i = \phi_0 + \sum_{j=1}^{M} \phi_{i,j}$$

Donde ϕ_0 representa el precio medio predicho en la muestra y $\phi_{i,j}$ es el valor SHAP que cuantifica la contribución exacta (positiva o negativa) de la característica j en el inmueble.

Además, investigaciones recientes en Barcelona han demostrado que aplicar algoritmos de agrupamiento (*k-means*) directamente sobre la matriz de valores SHAP extraídos de un modelo XGBoost permite identificar submercados inmobiliarios latentes de forma endógena, agrupando propiedades con perfiles de valoración estructuralmente homogéneos.



Diagnóstico de Errores Frecuentes en Modelado Tabular y Producción

1. Sesgo y Dispersión Residual en Extremos de Valor (Propiedades de Lujo / Submercados Atípicos):
    - *Problema:* Los modelos entrenados con funciones de pérdida globales cuadráticas (MSE) sufren de sesgo hacia la media y presentan residuos muy elevados en transacciones de extremo valor alto.
    - *Diagnóstico/Solución:* Transformar la variable objetivo mediante escala logarítmica $\ln(y)$ para estabilizar la varianza residual, aplicar remoción de *outliers* multivariados mediante Distancia de *Mahalanobis*, o entrenar estimadores *XGBoost* independientes por quantiles/submercados geográficos identificados mediante *clustering*.
2. Fuga de Información (*Data Leakage*) por Preprocesamiento Global:
    - *Problema:* Aplicar normalización, imputación o selección de variables sobre la totalidad del conjunto de datos antes de realizar la partición de entrenamiento/prueba (*train/test split*), contaminando el conjunto de prueba.
    - *Diagnóstico/Solución:* Aislar estrictamente la partición de prueba (*holdout test set*) y encapsular todas las transformaciones dentro de *pipelines* validados mediante validación cruzada anidada (*nested cross-validation*).
3. Desalineación por Autocorrelación Espacial al Usar Validación Cruzada Aleatoria:
    - *Problema:* La validación cruzada *k-fold* aleatoria estándar sobreestimará la capacidad de generalización fuera de muestra debido a la dependencia y proximidad geográfica entre registros.

- *Diagnóstico/Solución:* Implementar Validación Cruzada por Bloques Espaciales (*Spatial Block Cross-Validation*), donde zonas geográficas completas son excluidas en cada pliegue para simular la predicción real en ubicaciones no observadas.

Parte 2: Recursos interactivos de estudio

1. Mapa Mental Conceptual



2. Set de Tarjetas Didácticas (Flashcards)

Tarjeta 1: La Matriz Hessiana en XGBoost

- Pregunta (Anverso): ¿Cuál es la diferencia conceptual y matemática fundamental en la optimización de la función de pérdida entre el *Gradient Boosting Machine* (GBM) clásico y *XGBoost*?
- Respuesta (Reverso): El GBM clásico utiliza únicamente los gradientes de primer orden ($g_i = \frac{\partial l}{\partial \hat{y}}$) mediante descenso de gradiente. *XGBoost* incorpora una aproximación por la serie de *Taylor* de segundo orden, calculando simultáneamente las derivadas de primer orden (g_i) y los valores hessianos de segundo orden ($h_i = \frac{\partial^2 l}{\partial \hat{y}^2}$). Esto permite al algoritmo calcular con precisión analítica exacta la dirección y la magnitud óptima del paso de ajuste en los pesos de las hojas ($w_j^* = -\frac{\sum g_i}{\sum h_i + \lambda}$).

Tarjeta 2: Gestión de Valores Nulos (NaN)

- Pregunta (Anverso): ¿Cómo maneja *XGBoost* la presencia de datos perdidos o dispersos en vectores de entrada sin requerir imputación previa?
- Respuesta (Reverso): *XGBoost* aplica el algoritmo *Sparsity-aware Split Finding*. Durante la fase de construcción de cada nodo, el algoritmo asigna una dirección por defecto para todos los valores nulos o faltantes. Evalúa el beneficio de Ganancia (*Gain*) enviando los valores NaN a la rama izquierda y luego a la derecha, aprendiendo mediante el gradiente la ruta óptima que minimiza el error acumulado.

Tarjeta 3: Rol del Hiperparámetro Gamma (γ)

- Pregunta (Anverso): ¿Qué función operativa cumple el hiperparámetro gamma (γ) durante la construcción de los árboles de decisión en XGBoost?
- Respuesta (Reverso): gamma (γ) actúa como la constante de penalización por la adición de una nueva hoja en la función de regularización $\Omega(f_t)$. En la ecuación de Ganancia (*Gain*), γ funciona como un umbral mínimo estricto: si el diferencial de pérdida reducido al dividir un nodo no supera el valor de γ, la partición se descarta mediante un proceso de poda retroactiva (*pruning*).

Tarjeta 4: Prevención de Sobreajuste (L1 vs L2)

- Pregunta (Anverso): ¿Cómo se diferencian operativamente los hiperparámetros *reg_lambda* (λ) y *reg_alpha* (α) en el control de complejidad del modelo?
- Respuesta (Reverso): *reg_lambda* (λ) aplica regularización L2 (Ridge) sobre los pesos de las hojas w_j, comprimiendo sus magnitudes numéricas hacia cero en el denominador $\left(w_j^* = -\frac{\sum g_i}{\sum h_i + \lambda}\right)$ para suavizar las estimaciones. *reg_alpha* (α) aplica regularización L1 (Lasso), penalizando la suma de valores absolutos y forzando a que los pesos de hojas irrelevantes se vuelvan exactamente cero, promoviendo la dispersión del modelo.

Tarjeta 5: Explicabilidad Aditiva con SHAP

- Pregunta (Anverso): ¿Cómo transforma la integración de valores SHAP (*TreeExplainer*) la naturaleza de "caja negra" de un modelo *XGBoost* en el modelado tabular?
- Respuesta (Reverso): SHAP descompone de manera aditiva la predicción de una observación específica ($\hat{y}_i$) en la suma del valor esperado o media global del mercado (ϕ_0) más la contribución marginal exacta ($\phi_{i,j}$) atribuible a cada característica. Esto permite

expresar el impacto de cada atributo en unidades monetarias o proporcionales directas e interpretables, garantizando consistencia matemática y equidad en la atribución.

3. Cuestionario de Autoevaluación Formativa

Pregunta 1

Durante la optimización de un árbol de decisión en *XGBoost* bajo la función de pérdida del Error Cuadrático Medio ($l(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$), ¿cuál es el valor exacto del Hessiano de segundo orden (h_i) para cada observación i?

- **A)** $h_i = y_i - \hat{y}_i^{(t-1)}$
- **B)** $h_i = 2$
- **C)** $h_i = 2\left(y_i - \hat{y}_i^{(t-1)}\right)$
- **D)** $h_i = \lambda + \gamma$

Pregunta 2

Un arquitecto de Machine Learning observa que su modelo XGBoost obtiene un R^2 de 0,98 en el conjunto de entrenamiento, pero se degrada a un R^2 de 0,71 en el conjunto de validación. ¿Cuál de las siguientes combinaciones de ajustes en los hiperparámetros representa la solución técnica más adecuada para corregir este diagnóstico?

- **A)** Disminuir *max_depth*, incrementar gamma (γ) y aumentar reg_lambda (λ).
- **B)** Incrementar *max_depth*, reducir *min_child_weight* y fijar *learning_rate* en 1,0.
- **C)** Disminuir reg_alpha (α), eliminar la poda por gamma e incrementar *n_estimators*.
- **D)** Aumentar *learning_rate* a 0,5 y reducir *subsample* a 0,1.

Pregunta 3

Al evaluar un AVM basado en *XGBoost* sobre datos georreferenciados, ¿por qué la validación cruzada *k-fold* aleatoria convencional puede arrojar métricas de error engañosamente optimistas en comparación con la Validación Cruzada por Bloques Espaciales (*Spatial Block CV*)?

- A) Porque la validación aleatoria altera la distribución de los valores nulos procesados por el *Sparsity-aware Split Finding*.

- B) Porque la autocorrelación espacial entre propiedades cercanas hace que la partición aleatoria contamine el conjunto de validación con muestras geográficamente adyacentes a las de entrenamiento.
- C) Porque el algoritmo *TreeExplainer* de SHAP sólo puede ejecutarse sobre particiones estructuradas en bloques rectangulares.
- D) Porque el parámetro *colsample_bytree* no funciona correctamente cuando las observaciones están distribuidas aleatoriamente.

Solucionario Detallado del Cuestionario

- Respuesta Correcta a la Pregunta 1: B) $h_i = 2$
    - *Justificación Técnica:* La función de pérdida cuadrática se define como $l\left(y_i, \hat{y}_i^{(t-1)}\right) = \left(y_i - \hat{y}_i^{(t-1)}\right)^2$. La primera derivada respecto a la predicción $\hat{y}_i^{(t-1)}$ es el gradiente $g_i = -2\left(y_i - \hat{y}_i^{(t-1)}\right)$. Derivando por segunda vez respecto a $\hat{y}_i^{(t-1)}$, se obtiene la constante $h_i = \frac{\partial^2 l}{\partial\left(\hat{y}_i^{(t-1)}\right)^2} = 2$.
- Respuesta Correcta a la Pregunta 2: A) Disminuir *max_depth*, incrementar gamma (γ) y aumentar reg_lambda (λ).
    - *Justificación Técnica:* Una discrepancia significativa entre el rendimiento de entrenamiento (R^2 = 0,98) y validación (R^2 = 0,71) constituye un diagnóstico claro de sobreajuste (*overfitting*). Las acciones directas recomendadas consisten en restringir la capacidad estructural del árbol reduciendo *max_depth*, forzar la poda de nodos con baja ganancia mediante el aumento de gamma (γ), e incrementar la penalización L2 (*reg_lambda*) sobre los pesos de las hojas para favorecer la generalización.
- Respuesta Correcta a la Pregunta 3: B) Porque la autocorrelación espacial entre propiedades cercanas hace que la partición aleatoria contamine el conjunto de validación con muestras geográficamente adyacentes a las de entrenamiento.
    - *Justificación Técnica:* La Primera Ley de la Geografía establece que las observaciones cercanas espacialmente comparten patrones no observables y precios correlacionados. Una partición aleatoria coloca inmuebles de la misma calle o vecindario tanto en entrenamiento como en prueba, permitiendo que el modelo "memorice" la ubicación exacta. La Validación Cruzada por Bloques Espaciales

elimina esta contaminación al sostener zonas geográficas completas fuera del entrenamiento, midiendo la verdadera capacidad de generalización fuera de muestra del modelo.

Prompting: proceso de aprendizaje

1. Entender: Algoritmo de *Machine Learning XGBoost*

Objetivos de Aprendizaje y Conocimientos Previos

- Objetivos observables:
    1. Comprender el funcionamiento teórico del *Gradient Boosting* y cómo XGBoost lo optimiza mediante regularización y métodos algorítmicos avanzados.
    2. Identificar los hiperparámetros clave que controlan la complejidad y el rendimiento del modelo en problemas tabulares.
    3. Interpretar la estructura de árboles de decisión encadenados y el cálculo del gradiente de la función de pérdida.
- Conocimientos previos necesarios: Conceptos básicos de árboles de decisión, funciones de pérdida (como error cuadrático medio o entropía cruzada) y nociones de optimización por gradiente.

Bloque 1: Fundamentos de *Gradient Boosting* y la filosofía de *XGBoost*

- Concepto esencial: *Gradient Boosting* es una técnica de ensamblaje (ensemble) donde los modelos se entrenan de forma secuencial. Cada nuevo árbol intenta corregir los errores (residuos o gradientes) cometidos por la suma de los árboles anteriores.
- *XGBoost* (*Extreme Gradient Boosting*): Es una implementación altamente eficiente y optimizada de *Gradient Boosting* desarrollada por *Tianqi Chen*. Introduce mejoras sustanciales en velocidad de cómputo, manejo de valores perdidos y prevención del sobreajuste (*overfitting*).
- Ejemplo resuelto: Si estimamos el precio de una vivienda y el árbol 1 predice 200.000 € frente a un valor real de 220.000 €, el residuo (error) es +20.000 €. El árbol 2 no predice el precio total, sino que se entrena específicamente para predecir ese error de 20.000 €, afinando la predicción global.

- Analogía: Imagina un equipo de expertos en tasación inmobiliaria donde el primer experto hace una estimación general, el segundo corrige los fallos específicos del primero, el tercero corrige al segundo, y así sucesivamente. *Límite de la analogía:* Los expertos humanos discuten y colaboran en paralelo; XGBoost construye los modelos de manera estrictamente secuencial.

Bloque 2: Innovaciones Clave y Regularización en XGBoost

- Regularización L1 (α) y L2 (λ): A diferencia del *Gradient Boosting* tradicional, XGBoost incluye términos de penalización en su función objetivo para controlar la complejidad de los árboles (número de hojas y pesos), reduciendo drásticamente el sobreajuste.
- Optimización de la Función Objetivo: Utiliza una expansión de *Taylor* de segundo orden (empleando tanto el gradiente como la hessiana), lo que permite a la función matemática aproximarse de forma muy precisa independientemente de la función de pérdida elegida.
- *Sparseness-aware Split Finding*: *XGBoost* maneja de forma nativa los valores faltantes (*missing values*) asignando automáticamente una dirección por defecto (*default direction*) en cada nodo para optimizar el rendimiento sin imputaciones manuales previas.
- Ejemplo resuelto: Al evaluar una base de datos con variables incompletas sobre características constructivas, el algoritmo determina por sí mismo el camino óptimo para los registros que carecen de un dato específico (por ejemplo, antigüedad de la reforma), maximizando la ganancia de información.

Bloque 3: Hiperparámetros Esenciales

- *n_estimators*: Número de árboles (iteraciones de *boosting*) que se construirán.
- *learning_rate* (o *eta*): Tasa de aprendizaje que pondera la contribución de cada árbol nuevo para evitar que el modelo aprenda demasiado rápido (valores típicos entre 0,01 y 0,1).
- *max_depth*: Profundidad máxima permitida en cada árbol, controlando el nivel de interacción entre variables.
- *subsample* y *colsample_bytree*: Porcentajes de muestreo aleatorio de filas y columnas en cada árbol, aportando aleatoriedad y robustez frente al sobreajuste (similar al principio de *Random Forest*).

- Ejemplo resuelto: Configurar un modelo con *learning_rate* = 0,05 y *n_estimators* = 500 asegura que cada árbol aporte una pequeña corrección gradual, logrando una convergencia más estable y precisa que con tasas altas.

Errores Frecuentes

1. Confundirlo con *Random Forest*: Creer que los árboles se construyen de manera independiente y en paralelo. *Cómo reconocerlo:* En *XGBoost* los árboles son dependientes y se entrenan en cascada corrigiendo errores previos.
2. Ignorar el sobreajuste por exceso de profundidad (*max_depth* alto): Configurar árboles demasiado profundos sin regularización adecuada. *Cómo reconocerlo:* Excelente rendimiento en los datos de entrenamiento (*training*) pero métricas deficientes en validación o prueba (*test*).
3. No sintonizar la tasa de aprendizaje junto con el número de árboles: Aumentar *n_estimators* sin reducir el *learning_rate*, lo que dispara el tiempo de cálculo y satura el modelo.

Síntesis y Comprobación

XGBoost es una herramienta robusta y de alto rendimiento basada en árboles de decisión secuenciales optimizados mediante cálculo de gradientes de segundo orden y regularización avanzada, siendo el estándar de referencia en la resolución de problemas con datos tabulares.

Preguntas de Comprobación

1. ¿Cuál es la principal diferencia estructural entre el entrenamiento de *Random Forest* y el de *XGBoost*?
2. ¿Qué función matemática utiliza *XGBoost* para aproximar la función objetivo de manera eficiente?
3. ¿Qué efecto tiene la penalización por regularización L1 y L2 dentro de la función objetivo del algoritmo?

Respuestas a las Preguntas de Comprobación

1. *Random Forest* construye múltiples árboles de forma independiente y en paralelo (*Bagging*), mientras que XGBoost los construye de manera secuencial (*Boosting*), donde cada árbol nuevo aprende de los errores de los anteriores.
2. Una expansión en serie de *Taylor* de segundo orden, incorporando tanto las derivadas primeras (gradientes) como las derivadas segundas (hessianas).

3. Penalizan la complejidad del modelo (limitando la cantidad de nodos hoja y la magnitud de sus pesos), lo que previene el sobreajuste y mejora la generalización ante datos nuevos.

Recursos Interactivos Propuestos

- Mapa Mental: Estructura global conectando los conceptos de Ensamblaje, Gradiente de Taylor, Regularización y Manejo de Datos Faltantes.
- Tarjetas Didácticas (Flashcards):
    - *Tarjeta 1:* ¿Qué es learning_rate en XGBoost?
    - *Tarjeta 2:* ¿Cómo maneja XGBoost los valores perdidos?
    - *Tarjeta 3:* ¿Qué diferencia al *Gradient Boosting* clásico de XGBoost en su función objetivo?

2. Profundizar: Algoritmo de Machine Learning: XGBoost

1. Prerrequisitos y Dificultades Seleccionadas

- Prerrequisitos: Comprensión de árboles de decisión, cálculo diferencial básico (derivadas parciales y series de *Taylor*) y métricas de error y regularización en *machine learning*.
- Dificultades seleccionadas:
    1. La aproximación de segundo orden mediante la expansión de *Taylor* en la función objetivo.
    2. El manejo automatizado de valores perdidos (*sparsity-aware split finding*).
    3. El control del equilibrio entre regularización de hojas (γ, λ) y tasa de aprendizaje (*learning_rate*).

2. Análisis Detallado de Dificultades Conceptuales

Dificultad 1: Expansión de *Taylor* de Segundo Orden frente al Gradiente Estándar

- Por qué cuesta: Tradicionalmente, los modelos de *Gradient Boosting* optimizan la función de pérdida utilizando únicamente los gradientes de primer orden (los residuos). Comprender que XGBoost incorpora la matriz de derivadas segundas (Hessiana) a través de una aproximación cuadrática de Taylor genera confusión sobre por qué se necesita tanta complejidad matemática.

- Contraste conceptual:
    - *Gradiente de Primer Orden (GBM Clásico):* Solo mide la dirección y magnitud de la pendiente del error ($-\frac{\partial L}{\partial \hat{y}_i}$), tratando todos los errores con la misma curvatura teórica.
    - *Aproximación de Taylor de Segundo Orden (XGBoost):* Utiliza tanto el gradiente (primera derivada g_i) como la hessiana (segunda derivada h_i). Esto permite evaluar la curvatura de la función de pérdida, indicando al algoritmo qué tan rápido cambia la pendiente y otorgando un control milimétrico sobre el tamaño del paso en cada hoja.
- Caso comentado: Al calcular una función de pérdida compleja (como la pérdida logística o *Log Loss* en clasificación), el uso de la hessiana permite que la función objetivo de XGBoost se reduzca a una forma cuadrática exacta en términos de los pesos de las hojas (w_j), facilitando calcular analíticamente el peso óptimo de cada hoja ($w_j^* = -\frac{\sum g_i}{\sum h_i + \lambda}$) de forma directa y ultrarrápida.

Dificultad 2: Búsqueda de Cortes con Datos Dispersos (*Sparsity-aware Split Finding*)

- Por qué cuesta: Solemos asumir que los valores perdidos (*missing values* o NaN) requieren una fase previa de imputación (rellenar con la media, mediana o ceros). *XGBoost* rompe esta intuición al gestionar los valores nulos de manera nativa sin intervención manual.
- Contraste conceptual:
    - Imputación Estática Previa*:* Modifica el conjunto de datos antes de entrenar, lo que puede introducir sesgos artificiales o distorsionar la distribución real de las variables.
    - Dirección Predeterminada por Defecto en *XGBoost (Default Direction):* El algoritmo evalúa automáticamente en cada nodo de división qué rama (*left* o *right*) minimiza mejor la pérdida cuando faltan datos, asignando una dirección óptima temporal para los valores nulos en esa bifurcación específica.
- Caso comentado: En una base de datos de características inmobiliarias donde el campo "antigüedad de ascensor" está vacío para el 30% de los pisos antiguos, XGBoost no descarta las filas ni las rellena con ceros. Durante la búsqueda del punto de corte óptimo, el algoritmo evalúa si mandar los NaN a la izquierda o a la derecha maximiza la ganancia de información,

aprendiendo de manera implícita si la ausencia de dato correlaciona con un tipo específico de inmueble.

Dificultad 3: Regularización de Complejidad (γ, λ) frente a la Tasa de Aprendizaje (eta)

- Por qué cuesta: A menudo se confunden los parámetros que penalizan la estructura del árbol (γ para añadir hojas, λ para penalizar los pesos) con el hiperparámetro de contracción global (*learning_rate*), provocando un sobreajuste severo o un modelo subentrenado.
- Contraste conceptual:
    - *Parámetros de Estructura y Penalización (γ, λ):* Actúan a nivel interno de cada árbol individualmente. γ (gamma) establece la pérdida mínima que debe justificar la creación de un nuevo nodo hoja; λ aplica regularización L2 sobre los pesos predichos en las hojas para evitar que una sola hoja domine la predicción.
    - *Tasa de Aprendizaje (learning_rate /* eta*):* Actúa a nivel global del ensamblaje, multiplicando la contribución de cada árbol nuevo antes de sumarlo al modelo acumulado.
- Caso comentado: Si fijamos *max_depth* = 6 sin configurar un valor adecuado de γ o λ, el árbol crecerá creando divisiones excesivas para capturar ruido estadístico. Si además usamos un *learning_rate* = 1.0, el modelo memorizará el ruido del conjunto de entrenamiento. La combinación correcta requiere penalizar con $\gamma > 0$ y escalar la contribución con un learning_rate bajo (ej. 0.01).

3. Contraejemplo y Caso Límite

- Caso límite: ¿Qué ocurre cuando entrenamos *XGBoost* con un conjunto de datos extremadamente pequeño (por ejemplo, menos de 50 filas) y un número elevado de variables altamente correlacionadas?
- Análisis del cambio: En un escenario normal de *Big Data* tabular, la regularización de segundo orden y el submuestreo (*subsample*, *colsample_bytree*) garantizan una generalización robusta. Sin embargo, en un entorno de datos ultra reducidos, los árboles secuenciales tienden a sobreajustarse de inmediato porque cada nuevo gradiente intenta corregir los residuos de un conjunto minúsculo, provocando que el modelo sufra de *overfitting* extremo incluso con regularización alta.

- Lección práctica: XGBoost está optimizado para conjuntos de datos medianos y grandes; en *datasets* minúsculos, los modelos lineales simples o métodos de menor varianza suelen superar en estabilidad al boosting.

4. Tabla de Decisiones Operativas

Situación del Modelo	Criterio de Diagnóstico	Opción de Optimización Adecuada	Limitación o Riesgo
Sobreajuste (*Overfitting*): Alta precisión en entrenamiento, error elevado en validación.	*max_depth* excesivo, *learning_rate* muy alto, ausencia de penalización de hojas.	Reducir max_depth (ej. a 3-5), aumentar γ y λ, bajar *learning_rate* y activar subsample < 1.0.	Un modelo excesivamente conservador puede derivar en subajuste (*underfitting*).
Subajuste (*Underfitting*): Métricas deficientes tanto en entrenamiento como en validación.	*learning_rate* extremadamente bajo combinado con *n_estimators* insuficientes.	Incrementar *n_estimators*, subir ligeramente *max_depth* o aumentar la tasa de aprendizaje.	Aumenta el costo computacional y el riesgo de sobreajuste si se pasa el límite óptimo.
Presencia masiva de valores nulos (NaN): Pérdida de registros al realizar imputaciones manuales.	Uso de imputadores estáticos que distorsionan la varianza de las variables originales.	Aprovechar el *sparsity-aware split finding* nativo de *XGBoost* sin rellenar los nulos.	Requiere verificar que los nulos no oculten errores sistemáticos de captura de datos.
Tiempos de cómputo elevados en datasets masivos: Lentitud extrema en la búsqueda de cortes.	Búsqueda exacta de puntos de corte evaluando todos los percentiles de variables continuas.	Cambiar el parámetro *tree_method* a *hist* (*histogram-based algorithm*) o habilitar aceleración por GPU (*device='cuda'*).	Una aproximación basada en histogramas puede perder una fracción mínima de precisión en la división exacta.

5. Preguntas de Reflexión y Orientaciones de Respuesta

Preguntas de Reflexión

1. ¿Qué cambiaría en la estructura matemática de la función objetivo si eliminásemos el término de la Hessiana y utilizáramos únicamente el gradiente de primer orden?

2. ¿Por qué el algoritmo basado en histogramas (*tree_method*='hist') acelera drásticamente el entrenamiento de *XGBoost* en bases de datos masivas?
3. ¿De qué manera la penalización L2 (λ) sobre los pesos de las hojas previene que una anomalía aislada en el dataset distorsione la predicción final?
4. Si un modelo de *XGBoost* muestra un rendimiento perfecto en validación, pero falla estrepitosamente al implementarse en producción con nuevos datos, ¿qué hipótesis diagnóstica plantearías sobre la distribución de las variables?

Orientaciones de Respuesta

1. *Orientación 1:* Si se elimina la hessiana, el algoritmo pierde la información de curvatura (segunda derivada), volviendo a la formulación clásica de GBM donde no se puede calcular analíticamente el peso óptimo de la hoja de forma cuadrática exacta, requiriendo aproximaciones menos eficientes.
2. *Orientación 2:* Porque en lugar de ordenar y evaluar cada valor numérico de forma individual (lo que consume un tiempo computacional $O(N \log N)$), agrupa los valores continuos en contenedores (*bins*) discretos, reduciendo la complejidad del cálculo de cortes a $O(N)$.
3. *Orientación 3:* La penalización L2 actúa como un término de resistencia matemática en el denominador del cálculo del peso de la hoja ($w_j^* = -\frac{\sum g_i}{\sum h_i + \lambda}$), evitando que los pesos tomen valores extremos cuando la suma de las hessianas en una hoja es pequeña debido a muestras escasas.
4. *Orientación 4:* Apunta directamente a un problema de *Data Drift* (deriva de datos) o fuga de información (*data leakage*) durante el entrenamiento, donde el modelo aprendió patrones espurios del conjunto de validación que no se replican en el entorno real de producción.

6. Señales Observables de Comprensión

- Inferencias didácticas y distinción con fuentes: Las fuentes formales definen XGBoost a través de su formulación matemática y optimización computacional; las inferencias didácticas aplicadas en este informe traducen dichos formalismos a analogías operativas y tablas de decisión diagnóstica para facilitar su aplicación experta.
- Señales de dominio alcanzado:
    1. El alumno es capaz de explicar la diferencia entre la optimización basada en *Taylor* de segundo orden y el gradiente simple.

2. Identifica correctamente la función de los hiperparámetros de regularización (γ, λ) frente a la tasa de aprendizaje (eta).
3. Razona el comportamiento del algoritmo ante valores nulos sin requerir imputaciones externas previas.

Recursos Interactivos Propuestos

- Presentación Interactiva de Casos: Secuencia de diapositivas analizando paso a paso el cálculo de los pesos de las hojas mediante gradientes y hessianas en un caso práctico de estimación numérica.
- Infografía Comparativa: Esquema visual contrastando la búsqueda exacta de divisiones frente al método de histogramas (tree_method='hist') y el manejo nativo de valores perdidos.

3. Practicar: Algoritmo XGBoost

1. Habilidades a Practicar y Fuentes de Consulta

- Habilidades prácticas:
    1. Identificación y configuración de hiperparámetros de regularización y aprendizaje (*learning_rate*, *max_depth*, γ, λ).
    2. Diagnóstico operativo de modelos frente a problemas de sobreajuste (*overfitting*) y subajuste (*underfitting*).
    3. Aplicación de mecanismos nativos de *XGBoost* ante datos dispersos o con valores nulos (NaN).
- Fuentes de referencia recomendadas: Repasar los conceptos teóricos y tablas de decisiones de los bloques de Entender y Profundizar sobre la expansión de Taylor y el diseño de árboles secuenciales.

2. Ejemplo Resuelto y Justificación de Decisiones

- Enunciado de ejemplo: Un modelo de predicción de precios inmobiliarios muestra un R^2 de 0,99 en el conjunto de entrenamiento y de 0,60 en validación. El analista configuró *max_depth* = 12, *learning_rate* = 0,4 y sin parámetros de regularización.
- Procedimiento de solución:
    1. *Diagnóstico:* El desfase masivo entre entrenamiento (0,99) y validación (0,60) evidencia un sobreajuste severo (*overfitting*).

2. *Ajuste de hiperparámetros:* Se debe reducir la complejidad de los árboles fijando un límite menor (ej. *max_depth* = 4 o 5), incrementar los términos de penalización de hojas ($\gamma > 0$ y $\lambda > 1$) para frenar la memorización de ruido, y disminuir la contribución secuencial con un *learning_rate* = 0,05 acompañado de un mayor número de estimadores (n_estimators).

- Justificación de decisiones: Al limitar la profundidad y penalizar las hojas, se evita que los árboles crezcan arbitrariamente para capturar anomalías aisladas del *dataset* de entrenamiento.

3. Ejercicios Prácticos

Ejercicio 1 (Básico): Identificación de Arquitectura de Ensamblaje

- Consigna: Indica si la afirmación es verdadera o falsa y justifica brevemente: *"En XGBoost, los árboles de decisión se construyen de forma totalmente independiente y en paralelo, promediando sus resultados al final"*.
- Resultado esperado: Falso. *XGBoost* utiliza *Gradient Boosting*, lo que significa que los árboles se construyen de forma secuencial y en cascada, donde cada árbol nuevo aprende y corrige los residuos o gradientes dejados por los anteriores.
- Criterios de éxito: Distinguir correctamente entre *Bagging* (ej. Random Forest) y *Boosting* (ej. XGBoost).

Ejercicio 2 (Básico): Gestión de Valores Perdidos

- Consigna: Tienes un dataset tabular con un 20% de celdas vacías (NaN) en variables cuantitativas. ¿Qué método nativo de XGBoost evita que tengas que rellenar o imputar manualmente estos valores antes de entrenar?
- Resultado esperado: El algoritmo de búsqueda de cortes con conciencia de dispersión (*Sparsity-aware Split Finding*), el cual asigna automáticamente una dirección por defecto (*default direction*) en cada nodo.
- Criterios de éxito: Reconocer la capacidad nativa de *XGBoost* para procesar nulos sin sesgar artificialmente la distribución con imputaciones estáticas previas.

Ejercicio 3 (Intermedio): Diagnóstico de Hiperparámetros

- Consigna: Tu modelo de *XGBoost* presenta métricas deficientes tanto en el conjunto de entrenamiento como en el de validación (R^2 inferior a 0,5 en ambos). Tienes configurado

learning_rate = 0.001, *n_estimators* = 50 y un nivel de regularización extremo. ¿Cuál es el diagnóstico y qué ajuste inmediato realizarías?

- Resultado esperado: El modelo sufre de subajuste (*underfitting*) debido a una tasa de aprendizaje excesivamente baja combinada con muy pocos estimadores. Se debe incrementar el número de estimadores (*n_estimators*) o subir ligeramente el *learning_rate* y relajar la penalización excesiva.
- Criterios de éxito: Identificar el subajuste y proponer una solución orientada a dotar de mayor capacidad de aprendizaje al modelo.

Ejercicio 4 (Intermedio): Selección de Método de Cálculo Computacional

- Consigna: Vas a entrenar un modelo predictivo sobre una base de datos masiva que contiene más de 10 millones de filas. El tiempo de cómputo utilizando el método de búsqueda exacta de percentiles es prohibitivo. ¿Qué parámetro de configuración elegirías para acelerar el proceso manteniendo una precisión robusta?
- Resultado esperado: Configurar el parámetro de rendimiento *tree_method* = 'hist' (algoritmo basado en histogramas) o habilitar aceleración por hardware (*device* = *'cuda'*).
- Criterios de éxito: Conocer las alternativas de optimización computacional para grandes volúmenes de datos en *XGBoost*.

Ejercicio 5 (Transferencia): Auditoría de Modelo en Producción

- Consigna: Un modelo de clasificación de riesgo crediticio desarrollado con *XGBoost* funciona impecablemente en los entornos de prueba, pero al implementarse en producción comienza a fallar ante nuevos perfiles de clientes. ¿Qué conceptos analíticos (como derivada de segundo orden, deriva de datos o regularización) utilizarías para auditar la causa raíz y qué parámetro revisarías primero?
- Resultado esperado: Plantear una auditoría de *Data Drift* (deriva de datos) y revisar el parámetro de regularización L2 (λ) y la profundidad máxima (*max_depth*), ya que un modelo demasiado rígido o entrenado con datos sesgados no generaliza ante nuevas distribuciones en producción.
- Criterios de éxito: Vincular la teoría del algoritmo con problemas reales de despliegue y control de estabilidad.

Ejercicio 6 (Transferencia): Optimización de la Función Objetivo

- Consigna: Explica cómo el uso de la expansión en serie de *Taylor* de segundo orden (empleando gradientes g_i y hessianas h_i) permite a un ingeniero de datos calcular analíticamente el peso óptimo de una hoja en *XGBoost* frente a modelos de optimización tradicionales de primer orden.
- Resultado esperado: Explicar que la inclusión de la hessiana aporta información sobre la curvatura de la función de pérdida, permitiendo aproximar la función objetivo a una forma cuadrática exacta cuyo mínimo se obtiene analíticamente mediante la fórmula $w_j^* = -\frac{\sum g_i}{\sum h_i + \lambda}$.
- Criterios de éxito: Demostrar comprensión del soporte matemático avanzado de segundo orden que distingue a *XGBoost.*

4. Ayudas y Orientaciones (Consultar tras intentar los ejercicios)

- Pista 1 (Para Ejercicios 1 y 2): Recuerda que los algoritmos de *Boosting* dependen de los resultados anteriores, a diferencia de los bosques aleatorios. Asimismo, *XGBoost* no descarta los valores nulos, sino que evalúa por rama cuál minimiza el error.
- Pista 2 (Para Ejercicios 3 y 4): Si un modelo no aprende ni en entrenamiento, le falta capacidad o iteraciones (*n_estimators*, *learning_rate*). Para acelerar millones de filas, piensa en discretizar las variables continuas en contenedores (*bins*).
- Pista 3 (Para Ejercicios 5 y 6): La derivada primera indica la dirección; la derivada segunda (hessiana) indica la curvatura. Las desviaciones en producción suelen requerir auditar tanto el sobreajuste estructural como los cambios en la distribución de entrada.

5. Registro de Progreso del Alumno

Utiliza esta tabla para registrar tus intentos y planificar tus siguientes acciones de estudio:

Intento	Ejercicio / Tarea	Error Detectado	Corrección Aplicada	Siguiente Acción Recomendada
[1]	[Ej. Ejercicio 3]	[Ej. Confundí subajuste con sobreajuste]	[Ej. Revisé criterios de error en validación vs entrenamiento]	[Ej. Repasar tabla de decisiones operativas]
[ ]	[ ]	[ ]	[ ]	[ ]

4. Evaluar: Algoritmo XGBoost

1. Habilidades Evaluadas y Matriz de Relación

Las habilidades objeto de evaluación formativa abarcan la comprensión teórica de la optimización mediante series de *Taylor* y regularización, la capacidad de aplicación práctica e hiperparametrización, y la detección e interpretación de errores conceptuales comunes.

Habilidad Evaluada	Descripción Operativa	Bloque Relacionado / Fuentes
Comprensión Conceptual	Identificación de principios de *boosting* secuencial, expansión de Taylor y manejo de valores nulos.	Bloque 1 y Bloque 2 (Fundamentos e innovaciones)
Aplicación Práctica	Selección y ajuste de hiperparámetros (*learning_rate*, *max_depth*, γ, λ, subsample).	Bloque 3 y Tabla de Decisiones
Análisis de Errores	Diagnóstico de sobreajuste, subajuste y confusión entre arquitecturas de ensamblaje.	Errores Frecuentes y Casos Límite

2. Cuestionario de Evaluación Formativa

Bloque de Comprensión (Preguntas 1 a 4)

Pregunta 1: ¿Cuál es la diferencia estructural fundamental entre el entrenamiento de *Random Forest* y el de *XGBoost*?

- A) *Random Forest* entrena árboles de manera secuencial corrigiendo errores, mientras que XGBoost lo hace en paralelo.
- B) *Random Forest* construye múltiples árboles de forma independiente y en paralelo (*Bagging*), mientras que XGBoost los construye de manera secuencial (*Boosting*).
- C) Ambos algoritmos utilizan exactamente el mismo método de construcción en paralelo.
- D) XGBoost no utiliza árboles de decisión, sino redes neuronales profundas.

Pregunta 2: ¿Qué ventaja matemática aporta la expansión en serie de *Taylor* de segundo orden utilizada en la función objetivo de *XGBoost* en comparación con el *Gradient Boosting* clásico?

- A) Permite ignorar por completo las funciones de pérdida no lineales.
- B) Utiliza tanto el gradiente (primera derivada) como la hessiana (segunda derivada) para

evaluar la curvatura del error y calcular analíticamente el peso óptimo de las hojas.

- C) Elimina la necesidad de utilizar tasas de aprendizaje.
- D) Reduce el tiempo de cómputo a cero independiente del volumen de datos.

Pregunta 3: ¿Cómo gestiona *XGBoost* de manera nativa los valores perdidos (NaN) en las variables predictoras?

- A) Excluye automáticamente cualquier fila que contenga un valor nulo antes de iniciar el entrenamiento.
- B) Rustica los valores nulos reemplazándolos obligatoriamente por la media aritmética del *dataset*.
- C) Aplica una dirección por defecto (*default direction*) en cada nodo de división evaluando qué rama minimiza mejor la pérdida.
- D) Asigna un valor de cero por defecto a todos los campos vacíos.

Pregunta 4: ¿Qué función cumple el parámetro *learning_rate* (o *eta*) en el rendimiento global del modelo?

- A) Controla la profundidad máxima permitida en cada árbol de decisión.
- B) Pondera y escala la contribución de cada árbol nuevo antes de sumarlo al modelo acumulado para evitar aprendizajes demasiado rápidos.
- C) Determina el porcentaje de columnas que se seleccionarán aleatoriamente en cada árbol.
- D) Regula la penalización L1 para eliminar variables irrelevantes.

Bloque de Aplicación (Preguntas 5 a 8)

Pregunta 5: Si un modelo presenta un rendimiento excelente en el conjunto de entrenamiento, pero un error elevado en validación (*overfitting*), ¿cuál de las siguientes combinaciones de hiperparámetros es la más adecuada para corregirlo?

- A) Aumentar *max_depth* a un valor muy alto y subir el *learning_rate* a 1.0.
- B) Reducir *max_depth*, incrementar los parámetros de regularización (γ y λ), y disminuir el *learning_rate*.
- C) Eliminar los términos de regularización y aumentar el número de estimadores sin restricciones.

- D) Mantener los hiperparámetros intactos y duplicar el tamaño del conjunto de entrenamiento con datos sintéticos.

Pregunta 6: Necesitas entrenar un modelo de XGBoost en una base de datos masiva con millones de registros tabulares y buscas acelerar significativamente el tiempo de cómputo sin perder precisión crítica. ¿Qué parámetro de método de construcción de árboles deberías configurar?

- A) *tree_method = 'exact'*
- B) *tree_method = 'hist'* (algoritmo basado en histogramas)
- C) *tree_method = 'random'*
- D) *tree_method = 'linear'*

Pregunta 7: Quieres asegurarte de que ningún árbol individual dependa excesivamente de un subconjunto reducido de columnas o variables en el dataset. ¿Qué hiperparámetros debes ajustar?

- A) *n_estimators* y *learning_rate*
- B) *subsample* y *colsample_bytree*
- C) *max_depth* y gamma
- D) alpha y lambda exclusivamente

Pregunta 8: Al configurar la penalización por complejidad de los árboles, ¿qué efecto produce incrementar el parámetro γ (gamma)?

- A) Obliga al algoritmo a requerir una pérdida mínima mayor para justificar la creación de un nuevo nodo hoja, reduciendo la complejidad del árbol.
- B) Incrementa la velocidad de convergencia multiplicando el gradiente por una constante.
- C) Aumenta la profundidad máxima permitida en las ramas secundarias.
- D) Fuerza al modelo a ignorar los valores nulos del dataset.

Bloque de Análisis de Errores (Preguntas 9 a 12)

Pregunta 9: Un compañero de equipo afirma que ha utilizado *XGBoost* en un dataset diminuto de 30 filas y ha obtenido un modelo perfecto, por lo que se puede implementar directamente en producción. Según los criterios técnicos estudiados, ¿cuál es el error de diagnóstico en su afirmación?

- A) Confundió la tasa de aprendizaje con el número de estimadores.
- B) No aplicó una imputación manual previa de valores nulos.

- C) Incurrió en un riesgo severo de sobreajuste (*overfitting*) debido a que los algoritmos de boosting en datasets ultra pequeños memorizan el ruido en lugar de generalizar.
- D) Utilizó un método de cálculo de primer orden en lugar de segundo orden.

Pregunta 10: Si al revisar las métricas de un modelo observas que presenta un error elevado tanto en entrenamiento como en validación (*underfitting*), ¿qué diagnóstico es el más acertado?

- A) El modelo está sufriendo un exceso de regularización con un *learning_rate* extremadamente bajo o un número de estimadores insuficiente.
- B) El modelo tiene demasiada profundidad y se ha sobreajustado al ruido.
- C) Los valores perdidos no fueron eliminados correctamente del dataset.
- D) Se ha utilizado la hessiana de segundo orden de manera incorrecta.

Pregunta 11: Un analista asegura que la regularización L2 (λ) en *XGBoost* sirve exactamente para lo mismo que la tasa de aprendizaje (eta). ¿Por qué esta afirmación es incorrecta?

- A) Porque λ penaliza la magnitud de los pesos en las hojas para evitar valores extremos, mientras que eta escala globalmente la contribución del árbol completo al ensamblaje.
- B) Porque la regularización L2 solo se aplica a variables categóricas, mientras que eta se aplica a continuas.
- C) Porque λ es un parámetro dinámico y eta es estático.
- D) No es incorrecta; ambos parámetros hacen exactamente lo mismo en la práctica.

Pregunta 12: Durante la auditoría de un script de *XGBoost*, notas que se ha aplicado un preprocesamiento masivo imputando todos los valores nulos con la media antes de entrenar. ¿Qué implicación tiene esto frente a las capacidades nativas de *XGBoost*?

- A) El script fallará por incompatibilidad de librerías.
- B) Es un procedimiento obligatorio, ya que *XGBoost* no puede procesar datos sin imputación previa.
- C) Es un paso redundante, ya que *XGBoost* maneja de forma nativa los valores perdidos mediante la búsqueda de cortes por *sparsity-aware split finding*, aunque la imputación previa no invalida necesariamente el modelo.
- D) Provoca automáticamente un error de dimensiones en la expansión de Taylor.

3. Tareas Abiertas de Transferencia

Tarea 1: Diagnóstico y Rediseño de Hiperparámetros en Tasación Inmobiliaria

- Enunciado: Estás desarrollando un modelo predictivo de valoración inmobiliaria utilizando *XGBoost* sobre un dataset de 15.000 transacciones que contiene un 15% de valores nulos en variables constructivas y de calidades. El modelo inicial (*max_depth* = 10*, learning_rate* = 0,3, sin regularización) arroja un R^2 de 0,98 en entrenamiento, pero de 0,65 en validación.
- Consigna: Redacta un informe técnico justificando las causas del fallo, proponiendo los cambios específicos en los hiperparámetros y explicando cómo se comportará el manejo nativo de nulos frente a una imputación previa.

Tarea 2: Auditoría Forense de Algoritmos de Ensamblaje

- Enunciado: Un cliente te consulta sobre la conveniencia de utilizar *Random Forest* frente a *XGBoost* para un problema crítico de estimación masiva donde los errores pequeños deben penalizarse de manera exponencial y se requiere un control estricto de la curvatura del error.
- Consigna: Elabora una argumentación técnica basada en la formulación matemática de ambos algoritmos (primer orden vs. segundo orden mediante Taylor) para justificar cuál de los dos sistemas seleccionarías y qué riesgos operacionales vigilarías.

Rúbrica de Evaluación para Tareas Abiertas

Criterio de Logro	Nivel 1: En Desarrollo	Nivel 2: Competente	Nivel 3: Avanzado / Expert
Rigor Técnico y Conceptual	Confunde conceptos fundamentales (ej. confunde *boosting* con *bagging* o ignora el rol de la hessiana).	Explica correctamente los conceptos principales y justifica de forma coherente los hiperparámetros.	Domina la interacción entre la aproximación de Taylor, la regularización y la optimización del rendimiento en producción.

Criterio de Logro	Nivel 1: En Desarrollo	Nivel 2: Competente	Nivel 3: Avanzado / Expert
Capacidad de Diagnóstico	Atribuye los errores del modelo a causas genéricas sin relacionarlas con los hiperparámetros.	Identifica adecuadamente el origen del sobreajuste o subajuste y propone correcciones viables.	Realiza un diagnóstico forense completo integrando restricciones de datos, prevención de *overfitting* y eficiencia computacional.
Justificación Práctica	Las propuestas de optimización carecen de soporte técnico o contradicen los principios del algoritmo.	Argumenta lógicamente las modificaciones basándose en la estructura de los bloques de estudio.	Contextúala la solución a escenarios reales (como valoración o datos dispersos) demostrando visión crítica y aplicabilidad experta.

4. Ficha de Autoevaluación del Alumno

Completa esta ficha al finalizar la resolución del cuestionario y las tareas:

- Aciertos en el cuestionario: [Indicar número de aciertos / 12]
- Preguntas falladas (números): [Detallar cuáles fallaste]
- Conceptos implicados en los fallos: [Ej. Expansión de Taylor, hiperparámetros de regularización]
- Tipo de error principal cometidos: [Conceptual / Confusión de términos / Aplicación práctica]
- Dudas persistentes: [Anotar cualquier aspecto que requiera aclaración adicional]

Solucionario del Cuestionario

- Pregunta 1: B (*Random Forest* es *Bagging* en paralelo; *XGBoost* es *Boosting* secuencial).
- Pregunta 2: B (Usa gradiente y hessiana para la curvatura y cálculo analítico de pesos).
- Pregunta 3: C (Aplica dirección por defecto mediante *sparsity-aware split finding*).

- Pregunta 4: B (Escala y pondera la contribución de cada árbol nuevo).
- Pregunta 5: B (Reducir profundidad, subir regularización y bajar tasa de aprendizaje).
- Pregunta 6: B (*tree_method* = *'hist'* para datasets masivos).
- Pregunta 7: B (*subsample* y *colsample_bytree*).
- Pregunta 8: A (Requiere mayor ganancia para crear nuevas hojas, limitando la complejidad).
- Pregunta 9: C (Riesgo severo de sobreajuste en datasets ultra pequeños).
- Pregunta 10: A (Subajuste por regularización excesiva o tasa/estimadores insuficientes).
- Pregunta 11: A (Penaliza magnitud de pesos en hojas frente a escala global del árbol).
- Pregunta 12: C (Paso redundante dado el manejo nativo de nulos de *XGBoost*).

5. Planificar: Algoritmo XGBoost

1. Resumen de Datos y Contexto del Plan

- Datos recibidos: Estudio del Algoritmo XGBoost tras haber recorrido los módulos de comprensión, profundización, práctica y evaluación formativa.
- Plazo y disponibilidad (Supuesto editable): Al no haberse especificado un plazo ni disponibilidad de tiempo previos en la orden, se establece un plan provisional de 7 días con sesiones diarias de 30 minutos, optimizado para consolidar conceptos avanzados, solucionar errores frecuentes y afianzar la práctica.
- Diagnóstico inicial: Se parte de los prerrequisitos teóricos de árboles de decisión y optimización por gradiente, priorizando las dificultades detectadas en el análisis profundo (expansión de Taylor de segundo orden, manejo de datos dispersos/nulos y ajuste de hiperparámetros de regularización).

2. Priorización de Prerrequisitos y Errores

1. Expansión de Taylor de segundo orden (Gradiente + Hessiana): Prioridad alta por su complejidad matemática y su diferencia crítica frente al GBM clásico.
2. Hiperparámetros de regularización y aprendizaje (γ, λ,eta): Prioridad alta para evitar sobreajuste (*overfitting*) en escenarios reales.
3. Manejo nativo de valores perdidos (*Sparsity-aware split finding*): Prioridad media para optimizar el preprocesamiento de datos tabulares.

3. Estructura del Plan de Estudio por Sesiones (7 Días x 30 Minutos)

Sesión	Duración	Objetivo	Contenido / Sección Fuente	Actividad Práctica	Evidencia de Aprendizaje	Condición para Avanzar
Día 1	30 min	Consolidar fundamentos de *Gradient Boosting* vs XGBoost.	Bloque 1: Fundamentos y filosofía del algoritmo.	Resolver Ejercicio 1 básico (diferencia con Random Forest).	Explicar con propias palabras la construcción secuencial de residuos.	Distinguir sin errores Bagging de Boosting.
Día 2	30 min	Comprender la aproximación matemática de segundo orden.	Bloque 2: Expansión de Taylor (Gradiente y Hessiana).	Analizar cómo la hessiana calcula analíticamente el peso de las hojas.	Responder por qué se usan derivadas segundas.	Comprender el rol de la matriz de segundas derivadas.
Día 3	30 min	Dominar el manejo nativo de valores nulos.	Bloque 2: *Sparsity-aware split finding*.	Resolver Ejercicio 2 (gestión de `NaN` sin imputación previa).	Explicar el concepto de default direction en los nodos.	Entender el impacto de los nulos en la división de ramas.
Día 4	30 min	Ajustar hiperparámetros de regularización y aprendizaje.	Bloque 3: Hiperparámetros (`learning_rate`, γ, λ).	Resolver Ejercicio 3 (diagnóstico de subajuste y sobreajuste).	Ajustar correctamente γ y λ frente a un modelo con *overfitting*.	Diferenciar el rol local de γ / λ y global de `eta`.
Día 5	30 min	Optimizar rendimiento en bases de datos masivas.	Tabla de decisiones operativas (`tree_method = 'hist'`).	Resolver Ejercicio 4 (selección de métodos de cómputo).	Justificar el uso de histogramas frente a la búsqueda exacta.	Conocer las opciones de aceleración computacional.
Día 6	30 min	Resolver escenarios de transferencia y auditoría.	Tareas de transferencia y casos límite (datasets pequeños).	Resolver Ejercicio 5 (auditoría de modelo en producción).	Plantear soluciones ante *Data Drift* y desajustes de hiperparámetros.	Razonar la estabilidad del modelo en producción.
Día 7	30 min	Autoevaluación global y cierre del ciclo.	Cuestionario de evaluación formativa y Ficha de autoevaluación.	Completar el cuestionario de 12 preguntas y revisar aciertos/fallos.	Obtener un desempeño superior al 85% en el cuestionario.	Consolidación final del dominio de XGBoost.

4. Metodología de Estudio y Alternancia

- Distribución temporal: Cada sesión de 30 minutos se divide en 5 minutos de lectura activa, 10 minutos de recuperación de memoria (sin mirar apuntes), 10 minutos de resolución de ejercicios prácticos y 5 minutos de corrección y anotación en el registro.
- Espaciamiento: Los repasos de conceptos complejos (como la expansión de Taylor y la regularización) se reparten en días alternos para reforzar la retención a largo plazo.

5. Tarjetas de Repaso Rápido (Flashcards Clave)

- *Tarjeta 1:* ¿Qué ventaja aporta la hessiana en la función objetivo de *XGBoost*? -> Aproximación cuadrática precisa y cálculo analítico directo del peso de las hojas.
- *Tarjeta 2:* ¿Cómo maneja XGBoost los valores nulos (NaN)? -> Mediante búsqueda de cortes con conciencia de dispersión (sparsity-aware), asignando una dirección por defecto óptima por nodo.
- *Tarjeta 3:* ¿Qué diferencia existe entre γ y learning_rate? -> γ penaliza la creación de nuevas hojas en un árbol individual; *learning_rate* escala globalmente la contribución de cada árbol al ensamblaje.

6. Regla de Reajuste y Comprobación Final

- Regla de reajuste: Si en cualquier sesión fallas más de un ejercicio práctico o concepto clave, añade automáticamente 15 minutos adicionales a la siguiente sesión para repasar el bloque teórico correspondiente antes de avanzar.
- Comprobación final del objetivo: Al concluir el Día 7, serás capaz de explicar la arquitectura matemática de *XGBoost*, configurar adecuadamente sus hiperparámetros para evitar sobreajuste y auditar modelos en escenarios reales de datos tabulares.

Referencias Bibliográficas

Chen, T., & Guestrin, C. (2016).

- *XGBoost: A Scalable Tree Boosting System.*
- Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (San Francisco, CA, USA), pp. 785–794. DOI: 10.1145/2939672.2939785.

Kee, L., & Ho, C. (2025 / Publicado en 2026).

- *eXplainable Machine Learning for Real Estate: XGBoost and Shapley Values in Price Prediction.*
- Civil Engineering Journal (CEJ), Vol. 11, No. 5. DOI: 10.28991/CEJ-2025-011-05-022.

Schmid, J. (2026).

- *A Two-Stage Machine Learning Approach Using Linked Micro-, Socio- and Macroeconomic Data from Frankfurt am Main.*
- Real Estate (MDPI), Vol. 3, No. 3, 15. DOI: 10.3390/realestate3030015.

Sharma, H., Harsora, H., & Ogunleye, B. (2024).

- *An Optimal House Price Prediction Algorithm: XGBoost.*
- Analytics (MDPI), Vol. 3, pp. 42–58. DOI: 10.3390/analytics3010003.

Lundberg, S. M., & Lee, S.-I. (2017).

- *A Unified Approach to Interpreting Model Predictions.*
- Advances in Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA. arXiv: 1705.07874.

Rosen, S. (1974).

- *Hedonic Prices and Implicit Markets: Product Differentiation in Pure Competition.*
- Journal of Political Economy, Vol. 82, No. 1, pp. 34–55. DOI: 10.1086/260169.